JAVASCRIPT

RAPPELS

No notes on this slide.

JAVASCRIPT

Les fonctions

Les fonctions se déclarent avec le mot-clé `function`

```
function add(a, b){
    return a+b;
}
console.log(add(2,3)); // 5
```

En JavaScript, les fonctions sont des objets, qui peuvent être stockés dans des variables

```
const add = function (a, b){
    return a+b;
};
console.log(add(2,3)); // 5
```

On les appelle alors *fonctions anonymes*, car elles peuvent ne pas porter de nom.

En JS, les fonctions peuvent être déclarées en dessous de leur premier appel, cela n'a pas d'importance. On appelle cette propriété le *hoisting*. Quand un language possède des fonctions utilisables comme n'importe quel objet, passables en paramètres et retournables par d'autres fonctions, il est dit avoir des "first-class functions".

JAVASCRIPT

Les fonctions

Cela permet de passer des fonctions en paramètres d'autres fonctions

```
const mul2 = function (a){return 2*a;}
const squared = function (a){return a*a;}

function apply(array, func){
    const result = [];
    for(const x of array){
        result.push(func(x));
    }
    return result;
}

console.log(apply([1,2,3,4], mul2)) // 2,4,6,8
console.log(apply([1,2,3,4], squared)) // 1,4,9,16
```

Utiliser la notation `mul2` indique d'utiliser la fonction elle-même
Utiliser la notation `mul2(x)` appelle (execute) la fonction avec la valeur x

JAVASCRIPT

Les fonctions

Les *fonctions fléchées* (*arrow functions*) sont une notation encore plus compacte

```
const add = (a, b) => {
    return a+b;
};
console.log(add(2,3)); // 5
```

Si la fonction n'est composée que d'un `return`, on peut omettre les accolades et le `return`

```
const add = (a, b) => a+b;
console.log(add(2,3)); // 5
```

Toutes ces formes sont identiques, à l'exception de détails que nous n'aborderons pas dans ce cours

No notes on this slide.

JAVASCRIPT

Les fonctions

Les fonctions fléchées sont très utilisées en JavaScript, elles simplifient beaucoup la syntaxe.

```
function apply(array, func){
    const result = [];
    for(const x of array){
        result.push(func(x));
    }
    return result;
}

console.log(apply([1,2,3,4], a => 2*a)) // 2,4,6,8
console.log(apply([1,2,3,4], a => a*a)) // 1,4,9,16
```

En pratique, les tableaux possèdent une méthode `map` qui fait exactement l'effet de `apply`

```
console.log( [1,2,3,4].map(x => 2*x) ) // 2,4,6,8
console.log( [1,2,3,4].map(x => x*x) ) // 1,4,9,16
```


Si la fonction fléchée n'a qu'un paramètre, on peut même retirer les parenthèses autour du paramètre, comme dans l'exemple de cette slide.

JAVASCRIPT

Les fonctions - Ressources

Documentation officielle : https://developer.mozilla.org/fr/docs/Web/JavaScript/Guide/Functions

Tutoriel : https://www.w3schools.com/js/js_functions.asp

No notes on this slide.

JAVASCRIPT

Les objets

En JavaScript, un objet est un ensemble de propriétés qui possèdent une clé et une valeur

```
const myObject = {
    cle1 : "val1",
    cle2 : 35,
    cle3 : x => 2*x
}
```

Il s'agit de l'équivalent des dictionnaires en Python

No notes on this slide.

JAVASCRIPT

Les objets

Les clés peuvent être lues ou modifiées avec une notation utilisant un point ou des crochets

```
const myObject = {
    nombre: 8
};

console.log(myObject.nombre);       // 8
console.log(myObject['nombre']);    // 8

myObject.nombre = 9;                // Assigne 9
myObject['nombre'] = 9;             // Identique à la ligne précédente
```

Lire une clé qui n'existe pas retourne la valeur `undefined`.

Assigner une clé qui n'existe pas crée cette clé.

La notation avec un point est simple et ressemble beaucoup à d'autres languages orientés objet.
La notation avec des crochet permet d'accéder à un élément selon la valeur d'une variable. Par exemple :

```
const attr = 'nombre';
myObject[attr] = 9;
```

JAVASCRIPT

Les objets

On peut lister les clés d'un objet avec la boucle `for...in`

```
const myObject = {
    elem1: 8,
    elem2: "A"
};
for(const k in myObject){
    console.log(k + " : " + myObject[k]);
}
// Affichera :
// elem1 : 8
// elem2 : A
```

No notes on this slide.

JAVASCRIPT

Les objets

On peut définir des fonctions dans un objet de deux manières

```
const mesFonctions = {
    mult2 : function(x){           // Manière 1 : Assigner une fonction à une propriété
        return x*2;
    },
    mult3(x) {                     // Manière 2 : Directement écrire la fonction sans
        return x*3;                //             lui donner explicitement une clé
    }
};
console.log(mesFonctions.mult2(3)) // 6
console.log(mesFonctions.mult3(3)) // 9
```

Ces deux manières sont équivalentes

No notes on this slide.

JAVASCRIPT

Les objets

On appelle les variable stockées dans un objet des *attributs*

On appelle les fonctions stockées dans un objet des *méthodes*

```
const voiture = {
    poids: 1400,                    // Attribut
    marque: "BMW",                  // Attribut
    couleur: "blanc",               // Attribut
    sieges_chauffants: true,        // Attribut
    start_moteur() { /* ... */},    // Méthode
    stop_moteur() { /* ... */ },    // Méthode
    marche_arriere() { /* ... */ }  // Méthode
};
```

Comment faire si on souhaite créer plusieurs *voitures* dans notre programme sans se répéter ?

Comment faire pour que les méthodes d'un objet affectent ses attributs ?

No notes on this slide.

JAVASCRIPT

L'orienté objet et les classes

JavaScript permet de créer des constructeurs. Un constructeur est simplement une fonction destinée à créer des objets

```
function Point(x,y){
    this.x = x;
    this.y = y;
    this.display = () => console.log("("+this.x+";"+this.y+")");
}

const p = new Point(10, 20);    // Crée un objet nommé "p"
console.log(p.x);               // 10
p.display()                     // (10;20)
```

Le mot clé `this` est une référence vers le contexte courant, c'est-à-dire, dans ce cas, l'objet qui est créé par la fonction.

Le mot clé `new` indique de faire, à partir contexte de la fonction, un nouvel objet. C'est-à-dire, de passer tout ce qui a été assigné à `this` dans `p`

Un constructeur est simplement une fonction utilisée pour construire des objets. Elle n'a rien de particulier, à part qu'elle est utilisée dans ce but.

JAVASCRIPT

L'orienté objet et les classes

En JavaScript moderne, il existe un syntaxe plus proche des autres languages orientés objet.

Nous utiliserons cette syntaxe, plus lisible pour des objets complexes.

```
class Point{
    constructor(x,y){   // Le constructeur est une fonction qui se nomme toujours "construct
        this.x = x;     // C'est cette fonction qui sera appelée avec "new"
        this.y = y;
    }
    display(){
        console.log("("+this.x+";"+this.y+")");
    }
}

const p = new Point(10, 20);    // Crée un objet nommé "p"
console.log(p.x);               // 10
p.display()                     // (10;20)
```

Les syntaxes de cette slides et la précédente sont équivalentes

On dira "classe" pour parler du type conceptuel, mais pas d'un objet précis. Ainsi dans la "classe" Point possède les attributs x, y et la méthode display. On dira "objet" pour parler d'une instance de la classe. Ainsi la constance p est un objet de type Point, c'est-à-dire qui est une instance de la classe Point. Par convention, on commencera le nom des classes par une majuscule (Point) et on commencera le nom des objets par une minuscule (p).

La notation de cette slide est ce qu'on appelle un *sucre syntaxique*, c'est à dire une nouvelle notation pour quelque chose qu'on pouvait déjà faire avant, mais plus pratique. Elle n'apporte aucune fonction supplémentaire. JavaScript, au moment d'executer ce code, va le traiter exactement comme celui de la slide précédente, et n'importe quelle classe peut être écrite d'une manière ou de l'autre.

JAVASCRIPT

Les *getters* et les *setters*

Il est commun de vouloir modifier les attributs d'un objet, mais le faire directement peut créer des problèmes

```
class Disk{
    constructor(x, y, radius){
        this.x = x;
        this.y = y;
        this.radius = radius;
    }
    perimeter(){
        return 2*Math.PI*this.radius;
    }
}

const disk = new Disk(4, 5, 8);
disk.radius = -3;              // Rien n'empêche d'attribuer des valeurs ne faisant aucun s
console.log(disk.perimeter());  // Dans ce cas l'objet se comportera de manière inattendue
```

On aimerait vérifier que les attributs soient toujours cohérents

No notes on this slide.

JAVASCRIPT

Les *getters* et les *setters*

Pour cela, on peut utiliser les *getters* et les *setters*

```
class Disk{
    constructor(x, y, radius){
        this.x = x;
        this.y = y;
        this._radius = radius;
    }
    set radius(r){
        this._radius = r > 0 ? r : 0; // Ou n'importe quelle autre manière de gérer ce cas
    }
    get radius(){
        return this._radius;
    }
}

const disk = new Disk(4, 5, 8);
disk.radius = -3;               // Ce cas sera correctement géré
```

No notes on this slide.

JAVASCRIPT

Les *getters* et les *setters*

Cela permet aussi de simuler des attributs qui n'existent pas

```
class Disk{
    /* ... même contenu qu'avant ... */
    get area(){
        return Math.PI*this._radius*this._radius;
    }
    set area(a){
        this._radius = Math.sqrt(a/Math.PI);
    }
}

const disk = new Disk(4, 5, 8);
console.log(disk.area);     // Comme si area était bien un attribut
disk.area = 5;              // On peut également l'assigner
console.log(disk.radius);   // Et cela aura modifié la valeur de radius
```

L'utilisation des getters et de setters n'est pas du tout obligatoire, et certaines personnes écriraient clairement des méthodes `getArea()` et `setArea()`

No notes on this slide.

JAVASCRIPT

Les objets

Site officiel (très formel et complet):

Pour les objets : https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working_with_objects

Pour les classes : https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_classes

W3Schools:

Pour les objets : https://www.w3schools.com/js/js_object_advanced.asp

Pour les classes : https://www.w3schools.com/js/js_classes.asp

Cours de géoinformatique de Bachelor 2025 (GIN 2025) :

Page GitHub : https://github.com/corentin-junod/HEIG-VD-GIN-2025

Slides : https://github.com/corentin-junod/HEIG-VD-GIN-2025/tree/main/slides/pdfs

No notes on this slide.